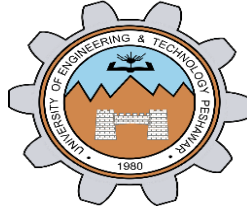


SMART NIGHT LAMP + OVERHEAT ALARM SYSTEM

PROJECT REPORT



SPRING 2026

Embedded Systems Lab 420L

Submitted by: **Zarina Miftah Durrani**

Faizan

Anees ur Rehman

Registration No. : **22PWCSE2126**

22PWCSE2134

22PWCSE2168

Class Section: **B**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Student Signature: _____

Submitted to:

Engr. Muhammad Shahzad

Submission Date:

2nd July, 2026

Department of Computer Systems Engineering

University of Engineering and Technology Peshawar

APPARATUS:

1. Arduino/Microcontroller Boards
2. Breadboard
3. Jumper Wires
4. USB Cable
5. Computer System with Arduino IDE
6. Power Supply

PROBLEM UNDERSTANDING:

The objective of this project is to design an embedded system that automatically controls a night lamp according to ambient light while simultaneously monitoring temperature for overheat protection.

The system models a real-world smart room lighting and safety monitoring system. During darkness, the lamp gradually turns on using PWM. If the surrounding temperature rises beyond a safe limit, a visual SOS warning is generated. If the temperature becomes critically high, both an LED and buzzer provide an emergency alarm. System status is continuously transmitted through UART.

DESIGN DECISIONS:

CHOICE	JUSTIFICATION
Arduino Uno (ATmega328P)	Selected because it provides a 10-bit ADC, hardware PWM, Timer0 and Timer2, UART communication, interrupt support, and sufficient GPIO pins. It is the standard development platform for this embedded systems lab and allows direct register-level programming.
LDR (Light Dependent Resistor)	Chosen to measure ambient light intensity for automatic lamp control. It is inexpensive, easy to interface using a voltage divider, and provides an analog output suitable for ADC conversion.
LM35 Temperature Sensor	Selected because it provides a linear output of 10 mV/°C, requires no external calibration, and directly interfaces with the ADC. It provides reliable temperature measurements for overheat detection.
LED1 (Night Lamp)	Used to simulate an automatic night lamp. Hardware PWM allows smooth brightness adjustment according to ambient light instead of abrupt ON/OFF switching, improving system behavior.
LED2 (Overheat Indicator)	Serves as a visual warning device during HOT and VERY_HOT conditions. It flashes in an SOS pattern to provide an easily recognizable emergency signal.
Buzzer	Provides an audible alarm during the VERY_HOT state. The buzzer ensures the warning can be heard even when the LEDs are not visible.
ADC Averaging & Hysteresis	Averaging 16 ADC samples reduces electrical noise and improves measurement stability. Hysteresis prevents rapid switching of the lamp and alarm when sensor readings fluctuate near threshold values.
UART Communication	UART at 9600 baud continuously sends sensor readings and system status to the Serial Monitor, making debugging and system monitoring easier during testing.

Bare-metal vs. Library Pairing for Each Subsystem

SUBSYSTEM	IMPLEMENTATION	JUSTIFICATION
ADC (LDR & LM35)	Bare-metal (ADMUX, ADCSRA, ADSC)	ADC registers are configured directly to select analog channels, reference voltage, and conversion timing. This demonstrates register-level control of analog inputs without using Arduino's analogRead().
Night Lamp PWM	Bare-metal Timer0 (TCCR0A, TCCR0B, OCRA)	Timer0 is configured in Fast PWM mode by directly writing to control registers. The PWM duty cycle is adjusted through OCRA to provide smooth LED brightness control.
SOS Alarm Timer	Bare-metal Timer2 Interrupt (TCCR2A, TCCR2B, OCR2A, TIMSK2)	Timer2 operates in CTC mode to generate a 1 ms interrupt. The interrupt drives the SOS blinking pattern while allowing the main program to continue reading sensors and updating outputs.
UART Communication	Bare-metal (UBRR0, UCSR0B, UCSR0C, UDR0)	UART registers are configured directly to transmit system status at 9600 baud. Register-level implementation satisfies the lab requirement and provides a deeper understanding of serial communication.
Sensor Processing	Custom C Functions	Sensor averaging, hysteresis, temperature conversion, and state-machine logic are implemented in software rather than using external libraries, providing full control over system behavior.

REGISTER-LEVEL WALKTHROUGH

1. ADC Configuration (LDR & LM35)

REGISTER	CODE / VALUE WRITTEN	BITS SET	WHAT IT CONFIGURES
ADMUX	(1<<REFS0)	REFS0 = 1	Selects AVCC as the ADC reference voltage. ADC channel selection (ADC0 for LDR and ADC1 for LM35) is updated before every conversion.
ADCSRA	(1<<ADEN) (1<<ADPS2) (1<<ADPS1) (1<<ADPS0)	ADEN=1, ADPS2:0=111	Enables the ADC and selects a prescaler of 128, producing an ADC clock of 125 kHz from the 16 MHz system clock.
ADCSRA	`ADCSRA	= (1<<ADSC)`	ADSC=1

2. Timer0 – PWM for Night Lamp

REGISTER	CODE / VALUE WRITTEN	BITS SET	WHAT IT CONFIGURES
DDRD	`DDRD	= (1<<PD6)`	PD6 = Output
TCCR0A	(1<<COM0A1) (1<<WGM01) (1<<WGM00)	COM0A1, WGM01, WGM00	Enables non-inverting Fast PWM mode on OC0A.
TCCR0B	(1<<CS01) (1<<CS00)	CS01, CS00	Sets Timer0 prescaler to 64. PWM frequency is approximately 976 Hz.
OCR0A	0–255	8-bit duty cycle	Controls LED brightness from 0% to 100% duty cycle.

3. Timer2 – SOS Interrupt

REGISTER	CODE / VALUE WRITTEN	BITS SET	WHAT IT CONFIGURES
TCCR2A	(1<<WGM21)	WGM21 = 1	Configures Timer2 in Clear Timer on Compare (CTC) mode.
OCR2A	249	Compare value	Generates a compare match every 1 ms using a 64 prescaler.
TIMSK2	(1<<OCIE2A)	OCIE2A = 1	Enables the Timer2 Compare Match A interrupt.
TCCR2B	(1<<CS22)	CS22 = 1	Selects a Timer2 prescaler of 64.

4. UART Configuration

REGISTER	CODE / VALUE WRITTEN	BITS SET	WHAT IT CONFIGURES
UBRR0H	0	Upper baud bits	Sets the upper byte of the UART baud-rate register.
UBRR0L	103	Lower baud bits	Configures UART for 9600 baud at a 16 MHz clock.
UCSR0B	(1<<TXEN0)	TXEN0 = 1	Enables the UART transmitter.
UCSR0C	(1<<UCSZ01) (1<<UCSZ00)	UCSZ01, UCSZ00	Selects an 8-bit data frame with one stop bit and no parity (8N1).
UDR0	UDR0 = c	Data register	Loads each transmitted character into the UART transmit buffer.

5. GPIO Configuration

REGISTER	CODE / VALUE WRITTEN	BITS SET	WHAT IT CONFIGURES
DDRB	`DDRB	= (1<<PB0) (1<<PB1)`	PB0, PB1 = Output
PORTB	`PORTB	= (1<<PB0)/PORTB &= ~ (1<<PB0)`	PB0
PORTB	`PORTB	= (1<<PB1)/PORTB &= ~ (1<<PB1)`	PB1

PINS CONNECTION TABLE

COMPONENT	ARDUINO PIN	ATMEGA328P PIN	CONNECTION	PURPOSE
LDR	A0	PC0 / ADC0	Connected as a voltage divider with a 10 kΩ resistor. LDR → +5V, junction → A0, resistor → GND.	Measures ambient light intensity.
LM35 Temperature Sensor	A1	PC1 / ADC1	Left pin → +5V, Middle pin → A1, Right pin → GND.	Measures ambient temperature.
LED1 (Night Lamp)	D6	PD6 / OC0A	D6 → 220 Ω resistor → LED anode; LED cathode → GND.	PWM-controlled night lamp.
LED2 (Alarm LED)	D8	PB0	D8 → 220 Ω resistor → LED anode; LED cathode → GND.	Visual SOS overheat alarm.
Buzzer	D9	PB1	Positive terminal → D9; Negative terminal → GND.	Audible alarm during VERY_HOT condition.
UART(TX)	D1	PD1 / TXD	Connected to USB Serial interface.	Sends system status to the Serial Monitor.
UART (RX)	D0	PD0 / RXD	Reserved for serial communication (unused).	Serial receive line.
Power Supply	5V & GND	VCC & GND	Powers the Arduino Uno and all connected components.	Provides operating power to the system.

Test Cases

TEST CASE	INPUT	EXPECTED BEHAVIOUR	OBSERVED RESULT
TC-1: Normal Operating Case	Room is well lit (LDR ≈ 640–816), temperature between 21°C–25°C .	LED1 remains OFF, LED2 and buzzer remain OFF. UART displays State:NORMAL.	LED1 stayed OFF, no alarm was activated, and the Serial Monitor displayed L:807 T:23C State:NORMAL.
TC-2: Boundary / Threshold Case	Cover the LDR so that the ADC value falls below DARK_ON (300) while temperature remains normal.	LED1 should gradually fade ON using PWM. LED2 and buzzer remain OFF.	LED1 brightness increased smoothly as the LDR value dropped below the threshold. No alarm was activated.
TC-3: Fault / Edge Case	Heat the LM35 sensor above 40°C (VERY_HOT_ON) .	LED2 should blink in an SOS pattern and the buzzer should activate. UART should display State:VERY HOT.	LED2 blinked in the SOS pattern, the buzzer sounded correctly, and the Serial Monitor displayed L:190 T:40C State:VERY_HOT.

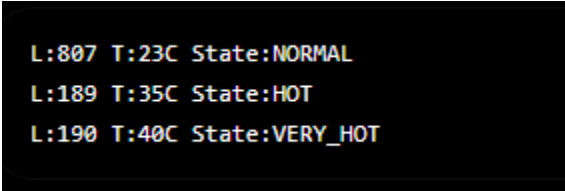
Results & Analysis

The Smart Night Lamp + Overheat Alarm system was tested under different lighting and temperature conditions to verify the functionality of each subsystem. The LDR and LM35 sensors successfully provided analog inputs through the ATmega328P ADC, while Timer0 PWM, Timer2 interrupts, and UART communication operated as intended.

The test results confirmed that the night lamp automatically responded to changes in ambient light. When the LDR reading dropped below the **DARK_ON (300)** threshold, LED1 gradually increased in brightness using hardware PWM. When the LDR reading increased above **DARK_OFF (500)**, the lamp faded OFF smoothly. The use of hysteresis prevented rapid ON/OFF switching when the sensor value fluctuated near the threshold.

Temperature monitoring also performed according to the implemented state machine. At temperatures below **35°C**, the system remained in the **NORMAL** state. When the temperature reached **35°C**, the system entered the **HOT** state and LED2 blinked in the programmed SOS pattern. At **40°C** and above, the system transitioned to the **VERY_HOT** state, activating both the SOS LED and the buzzer. The implemented hysteresis ensured stable transitions between states without unnecessary oscillations.

UART communication successfully transmitted the sensor readings and system status every second at **9600 baud**. The Serial Monitor displayed messages such as:



```
L:807 T:23C State:NORMAL
L:189 T:35C State:HOT
L:190 T:40C State:VERY_HOT
```

These outputs matched the expected operating conditions and verified that the ADC, PWM, Timer2 interrupt, and UART modules were functioning correctly.

Overall, the experimental results agreed with the expected behavior for all test cases. The system demonstrated stable sensor measurements through ADC averaging, smooth lamp brightness control using PWM, reliable alarm generation through timer interrupts, and accurate real-time status reporting via UART.

REFLECTION:

(a) Hardest Part

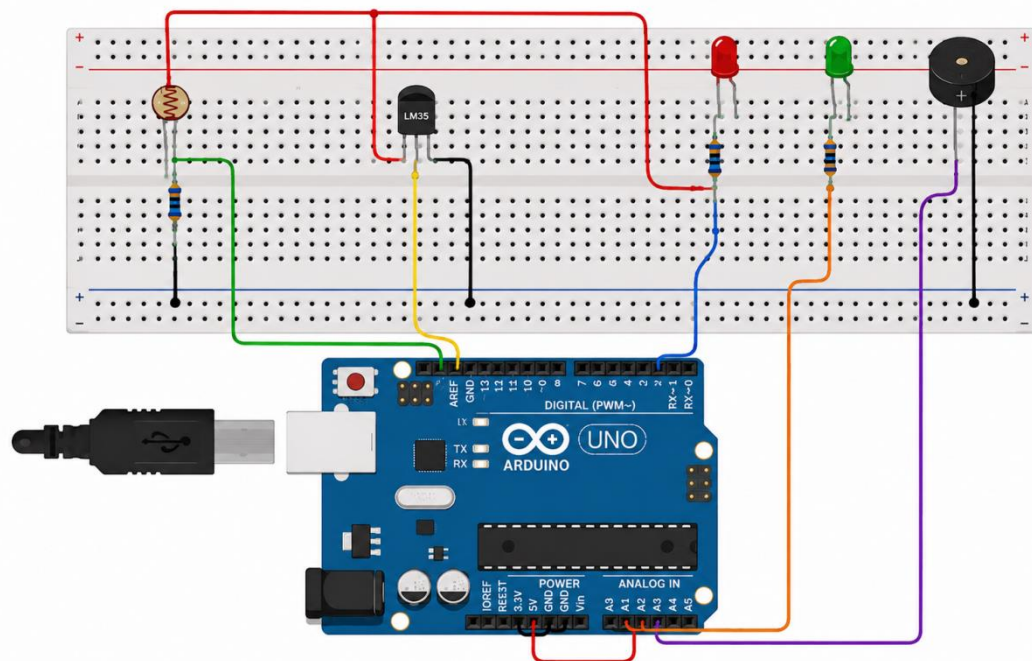
The most challenging aspect of this project was integrating multiple bare-metal peripherals to work together correctly. Configuring the ADC, Timer0 PWM, Timer2 interrupts, and UART communication simultaneously required careful register-level programming and debugging. In particular, ensuring that the Timer2 interrupt generated the SOS alarm pattern without affecting PWM brightness control or delaying sensor readings required a good understanding of interrupt-driven programming. Troubleshooting incorrect sensor readings caused by wiring issues and selecting appropriate hysteresis thresholds also took significant time during testing.

(b) What would you improve if you had more time?

Given additional development time, several improvements could be made to enhance the system:

- Add an LCD or OLED display to show temperature, light intensity, and system status without requiring a computer.
- Store user-defined threshold values in EEPROM so they are retained after power loss.
- Replace the LED night lamp with a relay or transistor driver to control a real household lamp.
- Add a cooling fan that automatically activates during the VERY_HOT state.
- Implement Bluetooth or Wi-Fi communication for remote monitoring through a mobile application.
- Improve power efficiency by placing the microcontroller into sleep mode during idle periods.

Circuit Diagram:



CODE:

```
/*  
*****  
* Project : Smart Night Lamp with Overheat Alarm  
* Platform: ATmega328P  
* Language: C (AVR-GCC)  
*/
```

*

* Description:

* -----

* This project implements:

* 1. Automatic night lamp using an LDR sensor.

* 2. Temperature monitoring using LM35.

* 3. PWM brightness control.

* 4. SOS LED warning when temperature is HOT.

* 5. SOS LED + Buzzer warning when temperature is VERY HOT.

* 6. UART status monitoring every second.

*

* Why C?

* -----

* C provides direct access to hardware registers, produces efficient code,

* has minimal overhead, and is the standard language for AVR bare-metal

* embedded programming.

*****/

#define F_CPU 16000000UL

#include <avr/io.h>

#include <avr/interrupt.h>

#include <util/delay.h>

#include <stdint.h>

#include <stdio.h>

/*-----

Hardware Pin Definitions

-----*/

#define LED2_PIN PB0 // SOS LED

#define BUZZ_PIN PB1 // Buzzer

/*-----

LDR Hysteresis Thresholds

-----*/

#define DARK_ON 300

#define DARK_OFF 500

/*-----

Temperature Hysteresis Thresholds

-----*/

#define HOT_ON 35

#define HOT_OFF 32

#define VERY_HOT_ON 40

#define VERY_HOT_OFF 37

```
/*-----  
    System Operating States  
-----*/
```

```
typedef enum  
{  
  
    STATE_NORMAL = 0,  
    STATE_HOT,  
    STATE_VERY_HOT
```

```
} system_state_t;
```

```
/*-----  
    Global Variables  
-----*/
```

```
system_state_t state = STATE_NORMAL;
```

```
volatile uint32_t ms_ticks = 0;
```

```
volatile uint8_t sos_enable = 0;
```

```
volatile uint8_t very_hot_enable = 0;
```

```
uint8_t lamp_active = 0;
```

```
/*=====
```

UART INITIALIZATION

```
=====*/
```

```
void uart_init(void)
```

```
{  
    /* Baud Rate = 9600  
       UBRR = 103 for 16 MHz */
```

```
    UBRR0H = 0;  
    UBRR0L = 103;
```

```
    /* Enable transmitter only */
```

```
    UCSR0B = (1 << TXEN0);
```

```
    /* 8-bit data  
       No parity  
       1 Stop bit */
```

```
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
```

```
}
```

```

/*=====
    UART SEND CHARACTER
=====*/

```

```

void uart_send_char(char c)
{
    /* Wait until transmit buffer becomes empty */

    while (!(UCSR0A & (1 << UDRE0)));

    UDR0 = c;
}

```

```

/*=====
    UART SEND STRING
=====*/

```

```

void uart_send_string(const char *str)
{
    while (*str)
    {
        uart_send_char(*str++);
    }
}

```

```

/*=====
    ADC INITIALIZATION
=====*/

```

```

void adc_init(void)
{
    /* REFS0 = 1
    AVCC used as reference voltage */

    ADMUX = (1 << REFS0);

    /* ADEN = Enable ADC
    ADPS2:0 = 111
    Prescaler = 128
    ADC Clock = 125kHz */

    ADCSRA =
        (1 << ADEN) |
        (1 << ADPS2) |
        (1 << ADPS1) |
        (1 << ADPS0);
}

```

```

/*=====
    ADC SINGLE READ
=====

```

```

uint16_t adc_read(uint8_t channel)
{
    channel &= 0x07;

    /* Select ADC channel */

    ADMUX = (ADMUX & 0xF0) | channel;

    /* Start conversion */

    ADCSRA |= (1 << ADSC);

    /* Wait until conversion finishes */

    while (ADCSRA & (1 << ADSC));

    return ADC;
}

/*=====
    ADC AVERAGE OF 16 SAMPLES
=====*/
uint16_t adc_read_avg(uint8_t channel)
{
    uint32_t sum = 0;

    for(uint8_t i=0;i<16;i++)
    {
        sum += adc_read(channel);
        _delay_ms(2);
    }

    return (uint16_t)(sum/16);
}

/*=====
    TIMER0 PWM INITIALIZATION
=====*/
void timer0_pwm_init(void)
{
    /* PD6 = OC0A output */

    DDRD |= (1 << PD6);

    /* Fast PWM
    Non-inverting mode */

```

```

TCCR0A =
    (1 << COM0A1) |
    (1 << WGM01) |
    (1 << WGM00);

/* Clock Prescaler = 64 */

TCCR0B =
    (1 << CS01) |
    (1 << CS00);

OCR0A = 0;
}

/*=====
    PWM DUTY UPDATE
=====*/
void set_lamp_brightness(uint8_t duty)
{
    OCR0A = duty;
}

/*=====
    TIMER2 CTC INITIALIZATION
=====*/
void timer2_ctc_init(void)
{
    /* CTC Mode */

    TCCR2A = (1 << WGM21);

    /* Compare value
       Generates interrupt every 1ms */

    OCR2A = 249;

    /* Enable compare interrupt */

    TIMSK2 = (1 << OCIE2A);

    /* Prescaler = 64 */

    TCCR2B = (1 << CS22);
}

/*=====
    TIMER2 INTERRUPT
=====*/

```

```

ISR(TIMER2_COMPA_vect)
{
    ms_ticks++;

    static uint16_t sos_timer = 0;
    static uint8_t sos_step = 0;

    if(!sos_enable)
    {
        PORTB &= ~(1<<LED2_PIN);
        PORTB &= ~(1<<BUZZ_PIN);

        sos_timer = 0;
        sos_step = 0;
        return;
    }

    sos_timer++;

    const uint16_t durations[] =
    {
        200,200,200,200,200,600,
        600,200,600,200,600,600,
        200,200,200,200,200,1000
    };

    const uint8_t outputs[] =
    {
        1,0,1,0,1,0,
        1,0,1,0,1,0,
        1,0,1,0,1,0
    };

    if(sos_timer >= durations[sos_step])
    {
        sos_timer = 0;
        sos_step++;

        if(sos_step >= 18)
            sos_step = 0;
    }

    if(outputs[sos_step])
    {
        PORTB |= (1<<LED2_PIN);

        if(very_hot_enable)

```

```

        PORTB |= (1<<BUZZ_PIN);
    else
        PORTB &= ~(1<<BUZZ_PIN);
    }
    else
    {
        PORTB &= ~(1<<LED2_PIN);
        PORTB &= ~(1<<BUZZ_PIN);
    }
}

/*=====
    AUTOMATIC LAMP CONTROL
=====*/
void update_lamp(uint16_t ldr_value)
{
    static uint8_t current_brightness = 0;

    uint8_t target = 0;

    if(!lamp_active && ldr_value < DARK_ON)
        lamp_active = 1;

    else if(lamp_active && ldr_value > DARK_OFF)
        lamp_active = 0;

    if(lamp_active)
    {
        if(ldr_value >= DARK_ON)
            target = 0;
        else
            target =
                (uint8_t)((((DARK_ON-ldr_value)*255UL)/DARK_ON));
    }

    if(current_brightness < target)
        current_brightness++;

    else if(current_brightness > target)
        current_brightness--;

    set_lamp_brightness(current_brightness);
}

/*=====
    TEMPERATURE STATE MACHINE
=====

```

```

void update_temperature_state(uint16_t tempC)
{
    switch(state)
    {
        case STATE_NORMAL:

            if(tempC >= VERY_HOT_ON)
                state = STATE_VERY_HOT;

            else if(tempC >= HOT_ON)
                state = STATE_HOT;

            break;

        case STATE_HOT:

            if(tempC >= VERY_HOT_ON)
                state = STATE_VERY_HOT;

            else if(tempC <= HOT_OFF)
                state = STATE_NORMAL;

            break;

        case STATE_VERY_HOT:

            if(tempC <= HOT_OFF)
                state = STATE_NORMAL;

            else if(tempC <= VERY_HOT_OFF)
                state = STATE_HOT;

            break;
    }

    if(state == STATE_NORMAL)
    {
        sos_enable = 0;
        very_hot_enable = 0;
    }
    else if(state == STATE_HOT)
    {
        sos_enable = 1;
        very_hot_enable = 0;
    }
}

```

else

```

{
    sos_enable = 1;
    very_hot_enable = 1;
}
}

/*=====
    STATE TO STRING
=====*/
const char* state_to_string(system_state_t s)

{
    switch(s)
    {
        case STATE_HOT:
            return "HOT";

        case STATE_VERY_HOT:
            return "VERY_HOT";

        default:
            return "NORMAL";
    }
}

/*=====
    MAIN PROGRAM
=====*/
int main(void)
{
    char buffer[64];

    /* Configure LED and buzzer as outputs */

    DDRB |= (1<<LED2_PIN) | (1<<BUZZ_PIN);

    adc_init();
    uart_init();
    timer0_pwm_init();
    timer2_ctc_init();

    /* Enable global interrupts */

    sei();

    uint32_t last_status_time = 0,

```

```
uart_send_string("Smart Night Lamp + Overheat Alarm Started\r\n");
```

```
while(1)
```

```
{
```

```
    uint16_t ldr_value = adc_read_avg(0);
```

```
    uint16_t lm35_adc = adc_read_avg(1);
```

```
    uint16_t tempC =
```

```
        (uint16_t)((lm35_adc * 500UL)/1024UL);
```

```
    update_lamp(ldr_value);
```

```
    update_temperature_state(tempC);
```

```
    if((ms_ticks-last_status_time)>=1000)
```

```
    {
```

```
        last_status_time = ms_ticks;
```

```
        snprintf(buffer,
```

```
            sizeof(buffer),
```

```
            "L:%u T:%uC State:%s\r\n",
```

```
            ldr_value,
```

```
            tempC,
```

```
            state_to_string(state));
```

```
        uart_send_string(buffer);
```

```
    }
```

```
    _delay_ms(20);
```

```
}
```

```
}
```

Results:

The Smart Night Lamp and Overheat Alarm system was successfully implemented on the ATmega328P microcontroller using Embedded C with bare-metal register-level programming. The system accurately monitored ambient light using the LDR sensor and temperature using the LM35 sensor. The ADC module successfully converted the analog sensor outputs into digital values for processing.

The night lamp responded correctly to changing light conditions. When the LDR reading dropped below the **DARK_ON (300)** threshold, the LED brightness increased gradually using

PWM generated by Timer0. When the ambient light increased and the LDR reading exceeded **DARK_OFF (500)**, the lamp smoothly faded OFF. The use of hysteresis prevented flickering when the sensor readings fluctuated around the threshold.

The temperature monitoring system also performed as expected. During normal operation (temperature below **35°C**), the system remained in the **NORMAL** state with all alarms disabled.

When the temperature reached **35°C**, the system entered the **HOT** state, activating the SOS blinking pattern on LED2. When the temperature increased to **40°C** or higher, the system transitioned to the **VERY_HOT** state, where both the SOS LED and buzzer were activated. The hysteresis values (**HOT_OFF = 32°C** and **VERY_HOT_OFF = 37°C**) ensured stable state transitions and eliminated unnecessary switching caused by small temperature fluctuations.

The Timer2 interrupt successfully generated the SOS blinking sequence without affecting the execution of the main program, demonstrating reliable interrupt-driven operation. In addition, UART communication configured through hardware registers transmitted the sensor readings and system status every second at **9600 baud**, allowing real-time monitoring through the Serial Monitor.

Conclusion:

Overall, the project strengthened practical knowledge of embedded systems by providing hands-on experience with memory-mapped registers, peripheral configuration, interrupt-driven programming, analog-to-digital conversion, and hardware-level software design. The successful implementation demonstrates the effectiveness of register-level programming for developing efficient and reliable embedded applications.